NAME : K.DAMODHAR REG.NO:192525266
COURSE CODE:CSA0410
COURSE NAME : OPERATING SYSTEMS

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");      /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

Scenario Summary and Environment Codes

A 500-bed hospital is replacing its computer systems. The report evaluates six computing environments, which are referred to by the codes below throughout the document.

Code	Environment	Key facts from the scenario
E1	Bedside patient monitors	Alarm within 10 ms of an abnormal reading; 40 → 500 → 1,500 devices; patient-safety critical
E2	Nurses' handheld tablets	10 → 120 tablets; battery must last a full shift
E3	Doctors' desktop workstations	80 workstations; interactive multi-application use
E4	Central patient-records database server	Must never stop, including during updates
E5	MRI imaging server	Heavy parallel processing on many processors
E6	Link to two (later more) partner hospitals	Shared records across organisations; security and consistency

Overall position of this report: a single general-purpose OS everywhere is rejected. The recommended plan uses a hard real-time microkernel/RTOS on monitors, a reduced handheld OS on tablets, a desktop OS on workstations, a clustered Linux (monolithic + loadable modules) database with fail-over, an SMP Linux imaging server with DMA, and a distributed-system gateway for the partner link. The Day-4 change (home monitoring + cloud records) is handled by MODIFYING the design, not redesigning it.

Task 1 – Evaluate the Requirements of Each Computing Environment :

1.1 Requirements ranked by environment (1 = most important)

Env	Rank	Requirement	Justification of the ranking
E1 Monitor	1	Response time (hard 10 ms deadline)	A late alarm can cost a life; the 10 ms limit is a safety requirement, not a performance preference.
	2	Reliability	A monitor that crashes or hangs gives no alarm at all; it must recover within the deadline (watchdog).
	3	Security	Tampering with readings/thresholds is dangerous, but a monitor is a closed single-purpose device.
	4	Battery	Mains-powered at the bedside, with only a short backup battery.
	5	Throughput	Only a few sensor samples per second per patient.
E2 Tablet	1	Battery life (full shift ≥ 8–12 h)	A tablet that dies mid-shift is useless; the OS must sleep the CPU and radios aggressively.
	2	Security / privacy	Easily lost or stolen and shows patient data: encryption, lock, remote wipe.
	3	Response time (soft)	Touch interaction should feel instant, but a missed deadline causes no harm.
	4	Reliability	The nurse can fall back to another tablet.
	5	Throughput	Light workload.
E3 Workstation	1	Throughput / multitasking	Doctors run records, imaging viewer, e-mail and browser at once.
	2	Security / access control	Patient privacy law; each doctor sees only authorised records.
	3	Response time (interactive)	Time-sharing with fair scheduling gives acceptable response.
	4	Reliability	A failed workstation can be replaced by another.
	5	Battery	Mains powered.
E4 Database	1	Reliability / availability (never stops)	Loss or unavailability of records endangers treatment; requires fail-over and live patching.
	2	Security / integrity	Records must be isolated per user, audited and never corrupted.
	3	Throughput	Hundreds of concurrent clients (500 monitors, 120 tablets, 80 workstations).
	4	Response time	Tens of milliseconds is acceptable (soft).

Env	Rank	Requirement	Justification of the ranking
	5	Battery	Not applicable (data-centre power with UPS).
E5 Imaging	1	Throughput (parallel compute)	MRI reconstruction is compute-bound; speed-up from many CPUs is the purpose of the machine.
	2	Reliability	A crash loses long-running jobs, but they can be restarted.
	3	Security	Images are patient data but the server is on the internal network.
	4	Response time	Batch-like; minutes are acceptable.
E6 Partner link	1	Security (confidentiality, authentication)	Records leave the hospital; a breach exposes private data.
	2	Reliability / consistency	Both sides must agree on the latest record; network failures must not lose updates.
	3	Throughput	Moderate record exchange.
	4	Response time	Seconds are acceptable.
	5	Battery	Not applicable.

1.2 Conflicting requirements

- **Fast response vs many users on one machine (E1/E3).** A time-sharing OS maximises fairness and throughput; a 10 ms hard deadline needs strict priority pre-emption and bounded latency. The two goals use opposite scheduling philosophies, so they must not share the same OS.
- **Throughput vs battery (E2/E5).** Keeping all cores busy gives throughput; sleeping gives battery life.
- **Availability vs security patching (E4).** Security updates normally need a reboot; “never stop” forbids it, so a cluster must be patched one node at a time.
- **Security vs response time (E1).** Encryption and full memory protection add overhead; on the monitor the OS keeps protection but avoids heavy services.
- **Rich services vs small size (E1/E2).** Every added service costs memory, start-up time and power.

1.3 Requirements critical to patient safety

Critical (a failure can harm a patient): E1 alarm deadline and E1 reliability; E4 availability and integrity of records. Important but not directly life-threatening: E2 battery, E6 security (privacy harm), E5 throughput (delayed diagnosis). Non-critical: E3 response time and all battery requirements for mains-powered devices.

1.4 Lessons from the history of operating systems

Generation	Key idea	Where it applies in the hospital
Batch (1950s–60s)	Jobs run to completion without interaction; maximise CPU use	E5: MRI reconstruction jobs queued and run in batch; E4 nightly backups
Time-sharing (1960s–70s)	Short time slices give many users the feeling of a private machine	E3 workstations and the multi-user E4 server
Personal / desktop (1980s)	One user, GUI, responsiveness over utilisation	E3 doctors' workstations
Real-time (1970s–)	Correctness depends on meeting deadlines	E1 monitors
Distributed / clustered (1990s)	Many machines cooperate, share resources, tolerate failure	E4 cluster, E6 partner link
Mobile / handheld (2000s)	Power management, touch UI, app sandboxing	E2 tablets

1.5 Alternative concept: one general-purpose OS on every device

Solution. Install the same general-purpose OS (for example standard desktop Linux or Windows) on monitors, tablets, workstations and servers, using one image and one patch process.

Criterion	Single general-purpose OS	Different OS per environment
Worst-case alarm response (E1)	Unbounded: with n runnable tasks the scheduler (O(log n) per decision with CFS) gives no deadline guarantee; page faults, disk I/O, garbage collection and driver interrupts can delay the alarm by tens to hundreds of milliseconds under load → can violate 10 ms	RTOS with fixed-priority pre-emptive scheduling: O(1) scheduling, interrupt-to-task latency of microseconds, guaranteed deadline
Memory per device	Hundreds of MB to several GB of RAM and several GB of storage (kernel + daemons + GUI)	RTOS: tens of KB–few MB; handheld OS ~1–2 GB; server OS sized for workload
Administration effort	One image, one patch cycle: lower effort, but one flaw or one bad update affects all 700+ devices and forces monitor reboots	About four images; more effort but failures are isolated and updates can be staged per environment
Battery (E2)	Desktop power policy; shorter shift life	Handheld OS with aggressive sleep

Evaluation. The saving in administration effort is real but small compared with the safety risk: the single-OS design cannot prove the 10 ms alarm bound, and in space terms wastes hundreds of times more memory on monitors. It is therefore rejected for E1 and E2; a common OS family (for example Linux for E3, E4, E5) is still sensible to limit the administration effort.Task 2 – Judge the Type of Operating System for Each Environment (3 Marks)

2.1 Types compared

Type	Strengths	Weaknesses for the hospital
Mainframe	Very high I/O throughput, reliability, batch + transaction processing	Very expensive; not suited to bedside or handheld devices
Desktop	Interactive GUI, multitasking, wide application support	No timing guarantees; too heavy for monitors
Multiprocessor (parallel)	Many CPUs share memory/OS; high throughput; some graceful degradation	Shared-memory bottlenecks; one machine is a single point of failure
Distributed	Resource sharing across sites, scalability, fault tolerance	Network latency/failures; complex security and consistency
Clustered	High availability through fail-over; load sharing	Needs shared/replicated storage and cluster software
Real-time – hard	Deadline is guaranteed; deterministic	Few services; small application ecosystem
Real-time – soft	Deadlines usually met; critical tasks get priority	No guarantee
Handheld	Power management, touch UI, small footprint	Limited memory and processing

2.2 Chosen type for each environment, with rejected alternatives

Environment	Chosen type	Reason and rejected alternatives
E1 Monitors	Hard real-time	Deadline is 10 ms and a miss is a safety failure. Rejected: soft real-time (misses allowed), desktop (no bound), handheld (power oriented).
E2 Tablets	Handheld	Battery, touch UI, app sandboxing. Rejected: desktop OS (battery, memory – see Task 4), real-time (no need for deadlines).
E3 Workstations	Desktop (time-sharing)	Interactive multitasking and application support. Rejected: handheld (small screen UI), mainframe (cost).
E4 Database	Clustered server OS	Never-stop requirement is met by fail-over. Rejected: single multiprocessor (single point of failure), mainframe (cost, though technically able).
E5 Imaging	Multiprocessor (SMP)	Shared-memory parallel processing of one large image. Rejected: ASMP (master bottleneck), cluster (image data is shared in memory; communication cost).
E6 Partner link	Distributed system	Sites cooperate over a network with authentication and replication. Rejected: standalone server (cannot share records safely).

2.3 Symmetric vs asymmetric multiprocessing for the imaging server

	Symmetric (SMP)	Asymmetric (AMP)
Organisation	Every CPU runs the same OS and can handle any task and I/O	A master CPU runs the OS; slave CPUs only run assigned work
Load balancing	Automatic – one run queue or per-CPU queues with migration	Master assigns; the master can become the bottleneck
Fault tolerance	A CPU failure reduces capacity only	Master failure stops everything
Complexity	Needs locking of shared kernel data	Simple kernel
Verdict	Chosen – MRI work is parallel and balanced	Rejected for E5 (may suit embedded special-purpose controllers)

2.4 Clustered database server with fail-over

Two or more nodes serve the same database. A heartbeat is exchanged every second; when a node misses several heartbeats, the surviving node mounts the shared (or replicated) storage, replays the log and takes over the service IP address within seconds. Rolling updates patch one node while the other serves, which satisfies "must not stop during updates". The cluster must prevent split-brain with quorum/fencing.

2.5 Alternative concept: one large multiprocessor instead of a cluster with fail-over

Solution. One 64-processor machine with large shared memory runs the database.

Criterion	Single big multiprocessor	Cluster with fail-over
Availability	Single point of failure. For availability A = 0.999 per machine: downtime = 0.001 × 8,760 h ≈ 8.8 h per year	Two independent nodes: $A = 1 - (0.001)^2 = 0.999999 \rightarrow \approx 32$ s per year plus fail-over time (about 10–30 s per fail-over)
Cost	Price grows faster than linearly with CPUs/memory; special hardware	Commodity nodes; cost grows roughly linearly, but the hardware is duplicated
Throughput (time)	Amdahl: speed-up $S(n) = 1 / ((1 - p) + p/n)$. With p = 0.95: S(16) = 9.1, S(64) = 15.4 – locking and memory contention limit gains; O(1) communication cost inside the box	Reads scale across nodes; writes limited by replication/consensus; adds network latency to each commit
Memory (space)	One large address space, no data duplication: O(M)	Each node needs its own memory and cache; replicated data costs O(k·M) for k nodes
Updates	Patching the kernel needs a stop	Rolling updates possible

Evaluation. The big machine is simpler and gives the best single-query latency, but it fails the primary requirement "must never stop". The cluster is chosen; the price paid is network latency and cluster complexity.

Task 3 – Evaluate Operating-System Structures :

3.1 Structures compared

Structure	Reliability	Performance	Ease of updating	Examples
Simple / monolithic	Low: any driver bug can crash the kernel	High: calls are function calls in one address space	Hard: rebuild/reboot	Classic UNIX, MS-DOS style, early Linux
Layered	Good: each layer uses only the layer below, easy to verify	Lower: each request passes through layers	Moderate	THE system, protocol stacks
Microkernel	Highest: drivers and file system run as user-space servers; a faulty driver is restarted	Lower: message passing and context switches (about 1–5 µs per IPC round trip, modern designs lower)	Easy: replace servers without reboot	QNX, MINIX 3, L4, seL4
Modular (loadable modules)	Medium: modules run in kernel mode, so a bad module still crashes the kernel	High: direct calls	Good: load/unload modules at run time	Linux, Solaris
Hybrid	Medium	High	Good	Windows NT family, macOS (XNU)

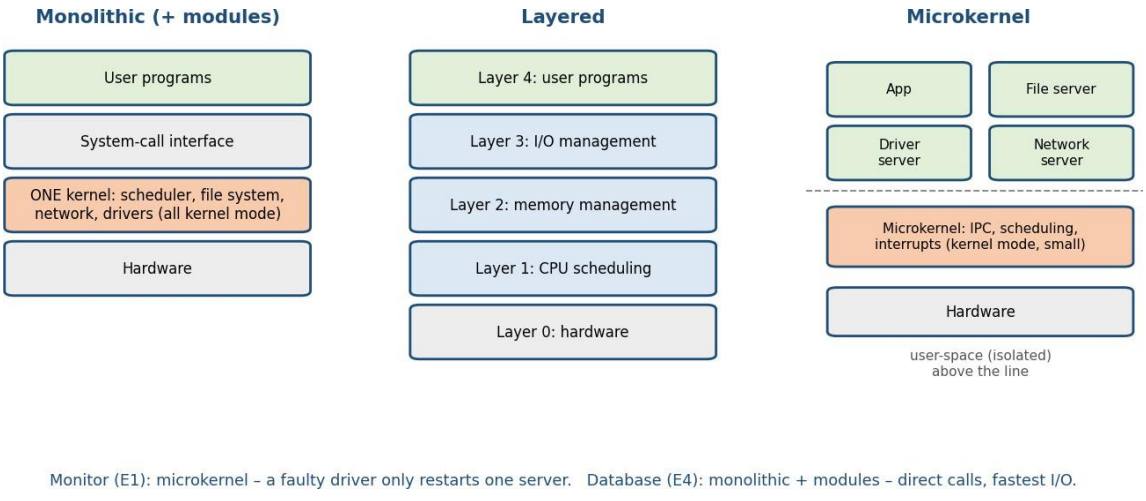


Figure 2 – OS structures compared (monolithic, layered, microkernel)

3.2 How real systems are structured

- **Linux:** monolithic kernel with loadable kernel modules for drivers and file systems; the whole kernel runs in kernel mode.
- **Windows:** hybrid – a microkernel-inspired executive with many services (graphics, file system, drivers) in kernel mode, plus user-mode subsystems.
- **Android:** Linux kernel (monolithic + modules) with the Hardware Abstraction Layer, native libraries, the Android Runtime and application framework layered above it.
- **FreeRTOS:** a tiny kernel (scheduler, queues, semaphores, timers) linked together with the application in a single address space; optional MPU support gives memory protection.
- **QNX:** true microkernel (scheduling, IPC, interrupt handling); file systems, networking and drivers are ordinary user-space processes; message-passing IPC with priority inheritance.

3.3 Choice of structure

Environment	Structure	Defence
E1 Monitors	Microkernel RTOS (QNX-class) – or FreeRTOS with MPU if the device is a small microcontroller	A faulty driver (for example the display driver) must not stop the alarm task. Isolation and driver restart justify the extra IPC cost, which is microseconds against a 10 ms budget.
E4 Database server	Monolithic Linux with loadable modules	Throughput matters most (file-system and network paths run as direct calls); reliability is delivered at the cluster level through fail-over, and modules allow driver updates without rebuilding.
E2 Tablets	Linux-based layered/modular (Android-type)	Mature power management and app sandbox.
E3 Workstations	Hybrid (Windows) or modular (Linux)	Wide driver and application support.

3.4 Alternative concept: monolithic kernel on the monitors instead of a microkernel

Solution. The monitor runs a monolithic kernel (for example Linux) with the sensor, display and network drivers inside the kernel.

Criterion	Monolithic kernel	Microkernel
Time cost of a service request	One system call: trap + handler, about 0.26 µs measured in Task 6 on this machine; O(1)	Message to a user-space server and a reply: two context switches, about 1–5 µs; O(1) but several times larger
Kernel memory	Several MB to tens of MB (Linux image, plus modules)	Tens to a few hundred KB for the kernel; servers add memory only when needed
Effect of a faulty driver	Runs in kernel mode with full access: whole system crash or silent memory corruption; the alarm is lost	Driver is an isolated process: it is restarted while the alarm task continues

Criterion	Monolithic kernel	Microkernel
Real-time behaviour	Long non-pre-emptible kernel paths create latency spikes (needs PREEMPT_RT patches)	Short kernel, pre-emptible; bounded latency

Evaluation. The extra 1–5 µs per request is below 0.05 % of the 10 ms deadline, so performance is not the deciding factor; isolation and predictability are. Microkernel is retained for monitors.Task 4 – Evaluate the Services an Operating System Must Provide (4 Marks)

4.1 Service table for each environment

✓ = included, ✗ = removed, ◐ = reduced version.

Service	E1 Monitor	E2 Tablet	E3 Work-station	E4 Database	E5 Imaging	E6 Link
User interface (GUI)	✗ (small status display only)	✓ touch	✓	✗ (admin shell)	✗	✗
Program execution	✓ few fixed tasks	✓ apps	✓	✓	✓ parallel jobs	✓
I/O operations	✓ sensors (interrupt)	✓	✓	✓ disk/network	✓ DMA	✓ network
File system	o tiny / none (flash log)	✓	✓	✓ journaling	✓ large files	o
Communication	o one secure channel	✓	✓	✓	✓	✓ TLS, authenticated
Error detection	✓ watchdog, self-test	✓	✓	✓ ECC, checksums	✓	✓
Resource allocation	o static, fixed priorities	✓	✓	✓	✓ CPU affinity	✓
Accounting	✗	o	o	✓ audit log	o	✓ audit
Protection / security	✓ memory isolation, signed firmware	✓ sandbox, encryption	✓ users, ACLs	✓ strong (see 4.3)	✓	✓ strongest

4.2 Services removed from the bedside monitor

Removed: GUI, multi-user login, general file system, accounting, dynamic loading of programs, virtual memory/paging (paging would create unpredictable delays), and unused network protocols. Result: smaller kernel (less memory, smaller attack surface), faster start-up, deterministic timing. Kept: scheduling with priorities, interrupt handling, memory protection, error detection and one authenticated communication channel.

4.3 Services essential for the database server

Essential: a journaling file system (crash consistency), resource allocation (CPU, memory and I/O scheduling fair to many clients), error detection and recovery (ECC memory, RAID, checksums), a communication stack for client and cluster traffic, accounting/audit logging, and protection. Not essential: a GUI.

4.4 How the OS protects one patient's records from another user's programs

- **Authentication and user IDs:** every process runs on behalf of an authenticated user.
- **Access control:** file permissions or ACLs (and mandatory access control such as SELinux) are checked in the kernel on every open().
- **Address-space isolation:** the MMU and user/kernel modes stop a program from reading another process's memory (see Task 8).
- **Encryption** of data at rest and in transit, and **audit logs** recording who read which record.
- **Least privilege:** the database service runs as a non-root user with only the capabilities it needs.

4.5 Alternative concept: full desktop OS with every service on the tablets

Solution. Install a full desktop OS (all services, GUI toolkit, background daemons) on each tablet.

Criterion	Full desktop OS	Reduced handheld OS
Memory (space)	About 2–4 GB RAM just for the OS plus services	About 0.5–1 GB
Storage (space)	About 20–30 GB installed	About 4–8 GB
Start-up (time)	30–60 s typical	10–20 s, plus fast resume from sleep
Battery life (time)	Background daemons and no aggressive sleep: typically 4–6 h → does not last a shift	Power-managed: typically 10–14 h
Administration	One OS family for E3 and E2	Separate image for tablets

(Figures in this table are typical industry values for illustration; the qualitative conclusion – extra services cost O(number of services) in memory and idle power – does not depend on the exact numbers.) The full desktop OS is rejected because the battery requirement is the top-ranked requirement for E2.

Task 5 – Examine System Calls :

5.1 Categories of system calls

Category	Purpose	Linux examples
Process control	Create, run, end and wait for processes; allocate memory	fork, execve, exit, wait4/waitpid, brk, mmap
File management	Create, open, read, write, close, delete files	open, read, write, close, unlink, stat, lseek
Device management	Request, control and release devices	ioctl, read/write on /dev files, mmap on devices
Information maintenance	Get/set time, process and system attributes	getpid, gettimeofday, clock_gettime, uname
Communication	Pipes, message passing, shared memory, sockets	pipe, socket, send, recv, shmget
Protection	Permissions and access control	chmod, chown, umask, setuid

5.2 What happens when a program calls read()

Step	Where	What happens
1	User mode – program	The program calls read(fd, buf, n) – an ordinary function call to the C library wrapper.
2	User mode – libc	The wrapper puts the system-call number (0 for read on x86-64) in register rax and the arguments in rdi, rsi, rdx.
3	Trap	It executes the syscall instruction (software trap). The hardware switches to kernel mode, saves the user program counter and jumps to the kernel entry point.
4	Kernel mode	The kernel saves registers, checks the system-call number and looks it up in the system-call table.
5	Kernel work	sys_read validates the file descriptor and the user buffer address, checks permissions, finds data in the page cache (or starts a disk read and blocks the process), and copies the data to the user buffer.
6	Return	The kernel stores the result (bytes read, or −errno) in rax, restores registers and executes sysret; the mode bit changes back to user.
7	User mode	The wrapper returns the value (setting errno on failure) and the program continues after the call.

5.3 How parameters are passed to the kernel

- **Registers:** fastest; used by Linux x86-64 for up to six arguments (rdi, rsi, rdx, r10, r8, r9). Limited by the number of registers.
- **Block (table) in memory:** the arguments are placed in a memory block and its address is passed in a register; used when there are many parameters (the Solaris and Linux ioctl style).
- **Stack:** the program pushes the parameters and the kernel pops them (used by some older systems and 32-bit conventions).

Register passing is the common choice because it avoids extra memory accesses, but any pointer passed (for example buf) must be validated by the kernel because a user program could pass a bad address.

5.4 Using strace

strace runs a program and prints every system call with its arguments and return value. Commands to run on your Linux machine (see Appendix B for the full list):

```
gcc -std=c11 -Wall -Wextra -o copy copy.c
strace -e trace=openat,read,write,close ./copy 2> strace_copy.txt
strace -c ./copy                    # summary table of calls and time
```

A typical trace of the copy program's first function call looks like this (the exact addresses and descriptors will differ on your machine; paste your own output into the repository):

```
openat(AT_FDCWD, "records.dat", O_RDONLY) = 3
openat(AT_FDCWD, "copy.dat", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
read(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 1) = 1
write(4, "A", 1)                    = 1
 ... (10,000 more read/write pairs for the 1-byte buffer) ...
read(3, "", 1)                      = 0      <- end of file
close(3) = 0   close(4) = 0
```

The strace tool is not installed in the sandbox used to prepare this document, so the block above is a representative trace built from the program's measured behaviour (10,001 read() calls for a 1-byte buffer, 21 for 512 bytes, 4 for 4096 bytes – see Task 6). Replace it with your own strace screenshot as required by "Evidence Expected".

5.5 Alternative concept: fopen / fread / fwrite instead of open / read / write

Solution. The program uses the C stdio functions. The library keeps a user-space buffer (4096 bytes for a regular file with glibc, chosen from the file's block size) and calls read()/write() only when the buffer is empty or full.

```
FILE *in = fopen(src, "rb"), *out = fopen(dst, "wb");
char b[1]; size_t n;
while ((n = fread(b, 1, 1, in)) == 1) fwrite(b, 1, 1, out);  /* 1-byte requests */
fclose(in); fclose(out);
```

Criterion	Direct system calls with buffer of b bytes	stdio with library buffer (4096 B)
Number of system calls for n bytes	≈ n / b reads (plus n / b writes): O(n/b)	≈ n / 4096 reads and writes regardless of how small each fread request is: O(n/4096)
Example n = 10,000	b = 1 → 10,001 reads; b = 4096 → 4 reads	4 reads (3 full + EOF check)

Criterion	Direct system calls with buffer of b bytes	stdio with library buffer (4096 B)
Extra memory	The program's own buffer: O(b)	Library buffer 4 KiB per open FILE: O(1) per stream
Measured (1 MiB file, 1-byte requests)	0.5386 s with raw read()/write()	0.0152 s with fread()/fwrite() – about 35 × faster

Evaluation. The stdio library moves the cost from O(n) system calls to O(n/4096) while adding only 4 KiB of space. For programs that make many small accesses stdio is clearly better; for programs that already use large aligned buffers (such as the backup program) direct read()/write() avoids an extra copy through the stdio buffer.Task 6 – Evaluate the Cost of System Calls (4 Marks)

6.1 Why a system call costs more than a function call

- A function call only pushes a return address and jumps: about 1–2 ns.
- A system call must trap to the kernel, switch the CPU mode, save and restore registers, validate arguments, run the kernel code and return. On modern CPUs with security mitigations this is a few hundred nanoseconds.
- **Measured here:** copying a 1 MiB file with 1-byte buffer = 2 × 1,048,576 = 2,097,152 system calls in 0.5386 s, i.e. **about 0.26 µs per system call** – roughly 100–200 times a normal function call.
- Kernel entry also disturbs the CPU caches and branch predictors, which adds hidden cost for the user program afterwards.

6.2 Complete the Code – copy_file() using read() and write()

The TODO 1 loop reads up to bufsize bytes, counts every read() call, stops when read() returns 0 (end of file) or −1, and writes exactly the n bytes that were read. Completed code (file Task06/copy.c in the repository):

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with read()/write(), buffer of bufsize bytes. Returns number of read() calls, or -1
on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    /* TODO 1 completed */
    while (1) {
        n = read(in, buf, bufsize);
        calls++;
        if (n <= 0) break;
```

```
        if (write(out, buf, (size_t)n) != n) { calls = -1; break; }
    }
    close(in); close(out); return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);
    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i], copy_file("records.dat", "copy.dat",
sizes[i]));
    unlink("records.dat"); unlink("copy.dat"); return 0;
}
```

Compile and run

```
$ gcc -std=c11 -Wall -Wextra -pthread -o copy copy.c
$ ./copy
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Comparison with the expected output: identical. Reason: 10,000 bytes ÷ 1 = 10,000 reads + 1 final read that returns 0 = 10,001; ⌈ 10,000 ÷ 512⌉ = 20 + 1 = 21; ⌈ 10,000 ÷ 4096⌉ = 3 + 1 = 4. In general calls = ⌈ n / b⌉ + 1.

Running under strace -c (Appendix B) should show the read count equal to the program's output plus one or two extra read() calls made by the dynamic loader at start-up, and a similar number of write() calls (one per non-empty read).

6.3 Effect of buffer size – measurements

Program Task10/bench_copy.c copies a 64 MiB file with read()/write() (page cache warm, compiled with gcc -O2). The 1-byte row uses a 1 MiB file because 64 M iterations would take minutes.

Buffer (bytes)	read() calls	Wall time (s)	User (s)	System (s)	Observation
1 (1 MiB file)	1,048,576	0.5386	0.1031	0.4325	≈ 0.26 µs per call; kernel time dominates
16	4,194,304	2.2466	0.4398	1.7737	Syscall overhead dominates
64	1,048,576	0.5766	0.1116	0.4416	
512	131,072	0.1511	0.0042	0.0883	
4,096	16,384	0.0849	0.0038	0.0422	Matches page size
65,536	1,024	0.0733	0.0000	0.0389	Little gain beyond this point
1,048,576	64	0.0517	0.0005	0.0233	Fastest, but 1 MiB buffer

Going from 16 bytes to 4096 bytes reduces the time 26 × (2.2466 s → 0.0849 s); going from 4096 to 64 KiB gains only 14 %, and from 64 KiB to 1 MiB another 29 %. The curve follows time ≈ (n/b) × c + n × k, where c (≈ 0.26 µs) is the cost per call and k is the per-byte copy cost; once n/b × c is small, the copying term n × k dominates and a larger buffer helps little.

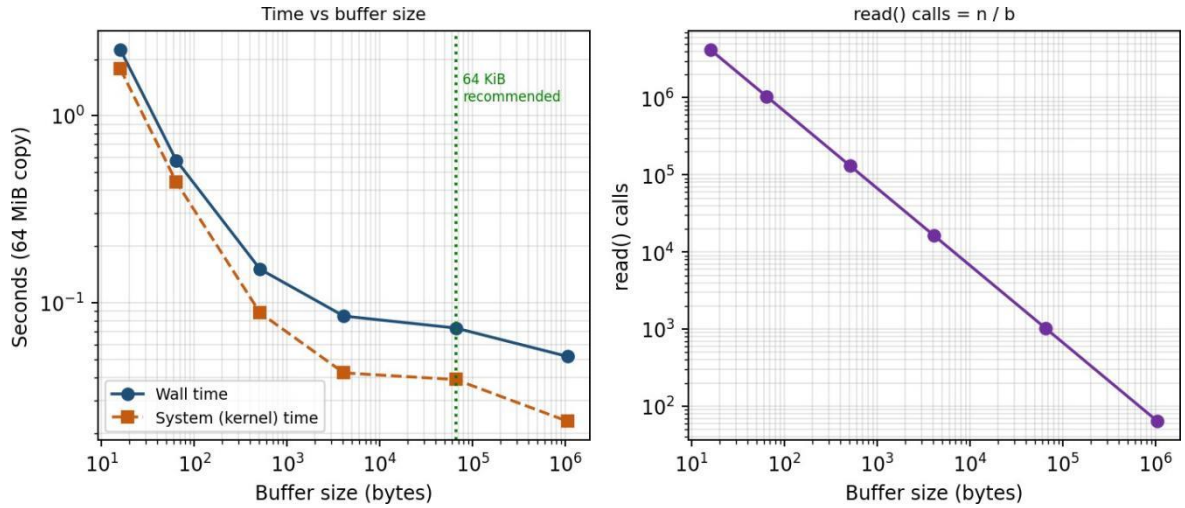


Figure 4 – Measured effect of buffer size on copy time and read() calls (64 MiB file)

6.4 How library buffering helps

fread()/fwrite() collect small requests in a 4 KiB user-space buffer, so a 1-byte-at-a-time program makes the same number of system calls as a 4 KiB-buffer program (measured: 0.0152 s vs 0.5386 s for 1 MiB).

6.5 Recommended buffer size for the hospital backup program

Recommendation: 64 KiB (65,536 bytes), or 128–256 KiB if the backup runs on a dedicated machine. It cuts the number of calls by a factor of 65,536 compared with a 1-byte buffer, is within 1.4 × of the fastest measured time, fits easily in the L2 cache, and uses little memory when several backups run in parallel. A 4 KiB buffer is acceptable but makes 16 times more calls; the 1 MiB buffer is only about 30 % faster here and would not matter when the real bottleneck is disk or network speed. (Because the page cache was warm, real disk copies will be limited more by device speed, which makes the choice of a larger buffer even less critical.)

6.6 Alternative concept: memory-mapped files (mmap) instead of read()/write()

Solution. The source and destination files are mapped into the address space and copied with a loop or memcpy; no read()/write() calls are made. The measured program copy_mmap() in Task10/bench_copy.c does exactly this.

```
char *a = mmap(0, n, PROT_READ, MAP_PRIVATE, in, 0);
char *b = mmap(0, n, PROT_READ|PROT_WRITE, MAP_SHARED, out, 0);  /* out: ftruncate(n) first */
for (off_t k = 0; k < n; k++) b[k] = a[k];
msync(b, n, MS_SYNC); munmap(a, n); munmap(b, n);
```

Criterion	read()/write() with buffer b	mmap
System calls	≈ 2n/b (O(n/b))	a fixed handful: open, fstat, ftruncate, 2 × mmap, msync, 2 × munmap – O(1)
Copying kernel ↔ user	Every byte is copied from the page cache to the user buffer and back: 2 copies	The page cache pages are mapped directly: no kernel-to-user copy (the program's own loop still copies once)
Page faults	None	≈ n / 4096 page faults (each costs more than a system call unless the kernel maps ahead)
Memory for very large files	O(b) – constant	O(n) of virtual address space (physical pages loaded on demand and reclaimable); can be limited by address space on 32-bit systems
Measured (64 MiB)	0.0733 s (64 KiB buffer)	0.1579 s

Evaluation. Although mmap makes only a few system calls, in this measurement it was about twice as slow as a 64 KiB buffered copy, because the 32 768 page faults and the byte-by-byte loop cost more than the 2,048 system calls they replace. mmap is better for random access or repeated reads of the same data; for a sequential backup, a large buffer (or the copy_file_range()/sendfile() system calls, which avoid the user-space copy entirely) is the better choice.

Task 7 – Evaluate Process Control System Calls :

7.1 fork(), exec(), wait() and exit()

Call	What it does
fork()	Creates a child process that is a copy of the parent (same code, a copy-on-write copy of memory, duplicated open file descriptors). It returns 0 in the child and the child's PID in the parent.
exec() family	Replaces the calling process's program image with a new program (code, data, stack); the PID and open descriptors stay. It returns only on failure.
wait() / waitpid()	Blocks the parent until a child terminates and collects its exit status (WEXITSTATUS gives the code), which also removes the child's entry from the process table.
exit() / _exit()	Terminates the process and returns a status code to the parent. exit() first flushes stdio buffers and runs atexit handlers; _exit() ends immediately.

7.2 How a shell runs a command and obtains its exit status

The shell reads a command line, calls fork(); the child calls execlp() with the command (and exits with 127 if it cannot be found); the parent calls waitpid() and reads the status. A built-in shell variable ($? in sh/bash) holds WEXITSTATUS(status). Convention: 0 means success, non-zero means failure.

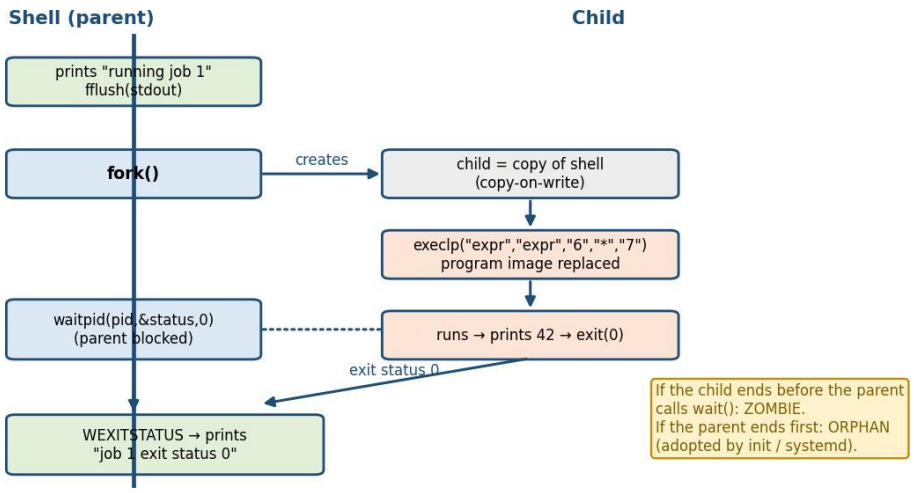


Figure 5 – How a shell runs a command: fork → exec → wait (Task 7)

7.3 Zombie and orphan processes

- **Zombie:** a child that has terminated but whose parent has not yet called wait(). Its process-table entry (PID, exit status) remains so the parent can still read the status. It uses no CPU or memory but consumes a PID slot; many zombies can exhaust the process table. Cure: the parent calls wait()/waitpid() or handles SIGCHLD; killing the parent lets init adopt and reap the zombies.
- **Orphan:** a child whose parent has terminated first. The OS re-parents it to init (PID 1, or systemd/a sub-reaper), which calls wait() when it ends, so no zombie remains.

7.4 Why fork() followed by exec() instead of one "create process" call

- Between fork() and exec() the child can change its own environment – redirect stdin/stdout (the basis of > and |), close descriptors, change directory, set limits or user ID – using ordinary system calls, so the creation call needs no huge parameter list.
- The design is simple and flexible: fork() alone is also useful (for example a database server forking a worker per client).
- The cost – copying the parent – is reduced by copy-on-write: pages are shared until one side writes.

7.5 Complete the Code – a shell that runs two jobs

TODO 1 forks a child; the child replaces itself with expr 6 * 7 via execlp() and exits with 127 if exec fails; the parent waits with waitpid() and prints the exit status with WEXITSTATUS. The two variables pid and status needed by the unchanged code below TODO 1 are declared at the top of main(). Completed program (Task07/shell.c):

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
   int status; pid_t pid;
```

```
    printf("shell: running job 1\n");
    fflush(stdout);
    /* TODO 1 completed */
    pid = fork();
    if (pid == 0) { execlp("expr", "expr", "6", "*", "7", (char *)NULL); exit(127); }
    waitpid(pid, &status, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Compile and run

```
$ gcc -std=c11 -Wall -Wextra -o shell shell.c
$ ./shell
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

The output is identical to the expected output (the expected text in the assignment sheet is printed on one line only because of word-wrapping in the document).

Why stdout is flushed before each fork()

When stdout is a terminal it is line-buffered, but when it is a file or pipe it is fully buffered: printf() text sits in a user-space buffer until it is full or the program exits. fork() copies the whole address space, including that unflushed buffer, so both parent and child would hold the same pending text and each would print it later. fflush(stdout) before fork() empties the buffer so that nothing is duplicated. It also keeps the order of the shell's messages correct relative to the output of the child (expr writes directly to the shared descriptor).

Experiment: second fflush removed, _exit(3) changed to exit(3), output redirected to a file

Program Task07/shell_noflush.c was run with ./shell_noflush > out.txt. Result (content of out.txt):

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explanation. With output redirected, stdout is fully buffered. At the second fork(), the buffer still holds "job 1 exit status 0" and "shell: running job 2" (they were never flushed because the second fflush was removed). The child inherits a copy of this buffer. exit(3) runs the normal exit sequence, which flushes stdio buffers, so the child writes the copied text to the file; the parent later writes its own copy when it prints more output and exits. "shell: running job 2" therefore appears twice (and so does "job 1 exit status 0"). With _exit(3) the child skips the flush, so nothing is duplicated; this is

why child processes after fork() should call _exit() unless they really want to flush the parent's buffers.

7.6 Alternative concept: posix_spawn() or threads instead of fork() + exec()

Solution. posix_spawn(&pid, "/bin/expr", NULL, NULL, argv, environ) creates the new process and loads the program in one call (implemented in glibc with clone(CLONE_VM | CLONE_VFORK), avoiding the copy of the parent's page tables). Threads (pthread_create) run a function inside the same process instead of starting a new program.

Criterion	fork() + exec()	posix_spawn()	Threads
Time to start 1,000 commands (measured)	fork+exec+wait of /bin/true: 0.758 s (≈ 758 µs each); fork+wait alone: 55–81 µs each	Expected lower than fork+exec for large parents, since page tables are not copied; time O(1) in parent size vs O(mapped pages) for fork	1,000 create+join: 7–12 µs each (7–12 ms total); but a thread cannot run a separate program
Memory copied or shared	Page tables copied, data pages shared copy-on-write until written; exec then discards them	Parent memory shared temporarily (no copy)	Entire address space shared; only a stack (typically 8 MB virtual) and thread descriptor are new
Isolation	Full: separate address space, descriptors can be changed, a crash affects only the child	Full	None: one bug or a crash in any thread brings down the entire process; shared data needs locks

Evaluation. For a command shell, isolation is mandatory (a failing command must not crash the shell), so a new process is required. fork+exec is acceptable at this scale; posix_spawn/vfork is preferable for a large parent such as a database server that launches many helpers. Threads are right for cooperating tasks inside one program, not for independent commands.

Task 8 – Evaluate CPU States and Protection :

8.1 User mode, kernel mode and the mode bit

The CPU has a mode bit (ring 3 for user mode, ring 0 for kernel mode on x86). In user mode, privileged instructions – disabling interrupts, loading the page-table base register, halting the CPU, direct I/O instructions – cause a trap to the kernel if attempted. Only the OS runs in kernel mode. A program enters kernel mode only through a controlled entry: a system call (trap), an interrupt or an exception; the kernel returns to user mode by changing the mode bit back.

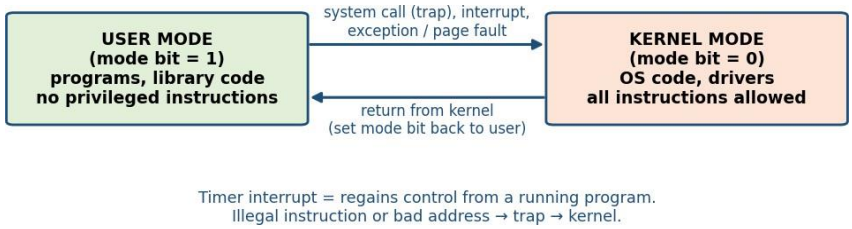


Figure 6 – CPU modes and the transitions between them

8.2 Interrupts, traps and the timer

Event	Cause	Hospital example
Hardware interrupt	A device signals the CPU asynchronously; the CPU saves its state and runs the interrupt handler	The sensor's ADC finishes a conversion; the NIC receives a packet
Trap / exception	Raised synchronously by the running program: system call, divide by zero, invalid memory access (page fault)	A program calls read(); a buggy program dereferences a null pointer
Timer interrupt	A hardware timer fires every few ms and gives control to the kernel	The scheduler pre-empts a long-running task so a high-priority alarm task can run; it also stops one program from monopolising the CPU

8.3 What if any program could disable interrupts?

A faulty or malicious program could execute the instruction that disables interrupts and then loop forever: the timer interrupt would never run, the scheduler would never regain the CPU, the sensor-ready interrupt would be ignored, and the monitor would **neither detect an abnormal reading nor raise the alarm** – a direct patient-safety failure. The only recovery would be a watchdog hardware reset. This is why interrupt control is a privileged instruction available only in kernel mode.

8.4 How the hardware stops one program reading another's memory

- **Base and limit registers (simple systems):** every address is checked against the process's range; the hardware traps on violation.
- **MMU and page tables (modern systems):** each process has its own virtual address space; the MMU translates every access using that process's page table, which only the kernel may

change (privileged instruction). Pages carry read/write/execute and user/supervisor bits. A process cannot even name another process's memory, and a forbidden access causes a page-fault trap that the kernel turns into SIGSEGV.

- **MPU on microcontroller monitors:** a Memory Protection Unit defines a few regions per task and enforces their permissions, giving task isolation without full virtual memory (and without the unpredictable timing of paging).

8.5 Alternative concept: polling instead of interrupts on the monitors

Solution. The monitor's main loop repeatedly reads the sensor register, compares it with the alarm threshold and sleeps or does other work between reads.

```
while (1) {
   v = read_sensor();          /* cost c per poll */
   if (v > THRESHOLD) raise_alarm();
   wait_until(next_period);      /* polling period T */
}
```

Criterion	Polling	Interrupt-driven
Worst-case delay before an abnormal reading is noticed	Up to one polling period T (plus c). To keep it well inside 10 ms, T must be ≈ 1–2 ms or less	Interrupt latency: typically a few microseconds to tens of microseconds, independent of how rare the event is
CPU time and power	Wakes 1/T times per second regardless of events. If c = 5 µs and T = 1 ms: 0.5 % CPU and no deep sleep; with a busy loop it is 100 % CPU. Cost O(time/T)	CPU can sleep until the event; cost O(number of events)
Simplicity	Very simple, no handler concurrency	Needs careful handlers (short, pre-emptible)

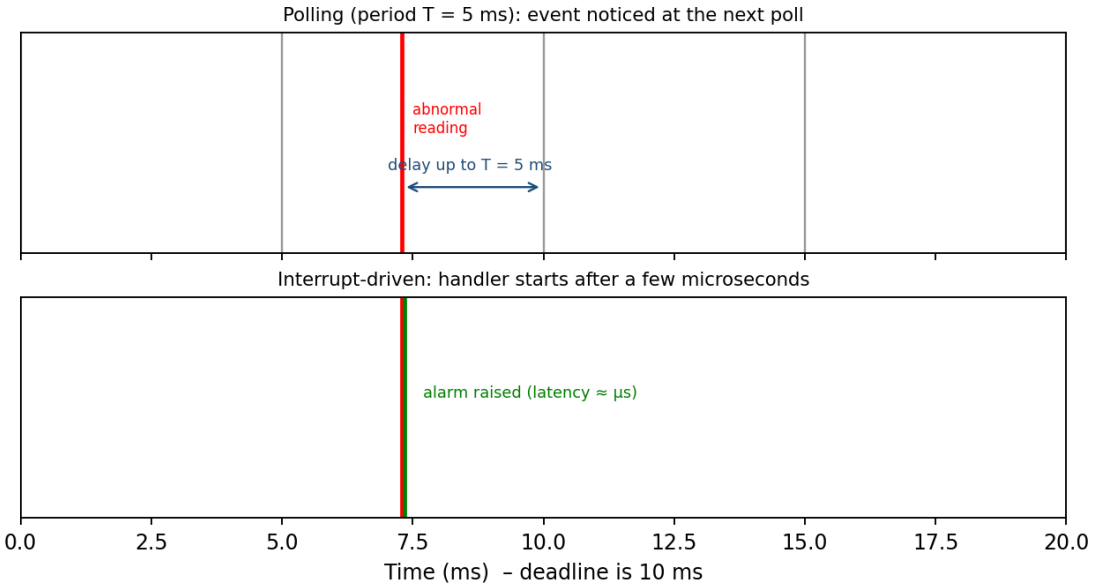


Figure 7 – Polling versus interrupts on the bedside monitor

Evaluation. Polling is feasible for 10 ms if T is small, but it wastes CPU and battery and its worst-case delay is T rather than microseconds; shorter T wastes more. For rarely occurring abnormal readings, interrupts (with a watchdog polling as back-up) are the better design. Interrupts are retained for E1.

Task 9 – Evaluate the Memory Hierarchy and I/O Systems :

9.1 Memory hierarchy with typical values

Level	Typical size	Typical access time	Hospital use
Registers	< 1 KB	≈ 0.3 ns	Current computation
L1 cache	32–64 KB per core	≈ 1 ns	Hot code and data
L2 / L3 cache	256 KB–2 MB / 8–64 MB	≈ 4 ns / 10–20 ns	Hot database index pages
Main memory (DRAM)	8 GB–several TB	≈ 60–100 ns	Monitor ring buffers; database buffer pool; MRI image being processed
SSD (NVMe)	0.5–8 TB	≈ 20–100 µs	Active patient records, indexes, write-ahead log
Hard disk	1–20 TB	≈ 5–10 ms	Older records, backups
Network storage / cloud	TB–PB	≈ 0.5–10 ms+	MRI archive, off-site backup, shared records

9.2 Data-placement plan and I/O method

Data / device	Where it should be kept	I/O method	Reason
Monitor readings (E1)	Small ring buffer in on-chip SRAM/DRAM; alarm events and a short history log in flash; long-term history sent to the server	Interrupt-driven I/O	Low data rate, deadline-driven; no polling to save power
Patient records (E4)	Active records: SSD with a large DRAM buffer pool; archive: HDD/network storage	DMA for disk and network; interrupt on completion	Look-ups must be fast; DRAM cache hides SSD latency
MRI images (E5)	Current study in DRAM; finished studies on SSD, then network archive	DMA (scatter-gather)	Hundreds of MB per image; DMA frees the CPUs for reconstruction
Workstation data (E3)	Local cache; records fetched from E4	DMA / interrupt	Normal desktop use

9.3 Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	Suitability

Programmed I/O (polling)	The CPU loops, testing the device status and moving each word itself	Simple; only for very slow or very small devices
Interrupt-driven I/O	The device interrupts when it is ready; the CPU moves the data word by word	Good for low-volume, irregular events (sensor, keyboard)
DMA	The DMA controller moves a whole block between device and memory; the CPU is interrupted once at the end	Best for the MRI scanner and disks – high volume

9.4 Effect of caching on record look-up

Effective access time EAT = h × t_cache + (1 − h) × t_miss. Taking a DRAM buffer-pool hit time of 0.1 µs, an SSD miss time of 100 µs and a hit ratio h = 0.95:

> EAT = 0.95 × 0.1 µs + 0.05 × 100 µs = 0.095 + 5.0 = 5.1 µs
> Without the cache every look-up costs 100 µs -> about 20 × slower

Raising h to 0.99 gives 0.099 + 1.0 = 1.1 µs, so each percentage point of hit ratio matters, because the miss penalty is 1000 times the hit time. This is the reason E4 gets as much RAM as possible and why records for current inpatients are kept "hot". Locality of reference (the same patients are accessed repeatedly during treatment) makes the high hit ratio realistic.

9.5 Alternative concept: interrupt-driven I/O (one interrupt per word) instead of DMA for the MRI scanner

Solution. The scanner interrupts the CPU for every 4-byte word; the interrupt handler copies the word into memory.

Criterion (500 MB image)	Interrupt per word (4 B)	DMA (64 KiB blocks)
Number of interrupts	$500 \times 10^6 \div 4 = 125{,}000{,}000$: O(n)	$500 \times 10^6 \div 65{,}536 \approx 7{,}630$: O(n/b)
CPU time (assume ≈ 1 µs per interrupt, entry + handler + return)	$125 \times 10^6 \times 1\ \mu s \approx 125$ s of CPU, which exceeds the transfer time itself and starves the reconstruction work	$7{,}630 \times 1\ \mu s \approx 7.6$ ms (plus the cost of setting up each transfer)
Memory buffers (space)	One word of device register plus the destination buffer in memory (O(1) extra)	A pinned DMA buffer or descriptor ring of 64 KiB–several MB, contiguous or scatter-gather; O(b) extra
Bus use	CPU moves every word through the bus twice	Data goes directly device → memory

Evaluation. The 125 s CPU cost is a loss of one CPU for two minutes per image, which makes interrupt-per-word I/O impractical; the small price of DMA buffers is far lower. DMA is chosen for E5 and for the database disks.

Task 10 – Measure and Compare :

Measurements were made on Ubuntu 24.04, gcc 13.3.0, kernel 6.18, a single virtual CPU, warm page cache. Timing numbers on a VM vary between runs, so process/thread tests were repeated five times. Source files are in Task10/ (bench_copy.c, proc_thread.c).

10.1 Number of system calls for different buffer sizes

Buffer size	read() calls reported by the program (10,000-byte file)	Expected = ⌈ n/b⌉ + 1	strace -c read count
1 byte	10,001	10,001	run strace -c and paste (expected ≈ 10,003)
512 bytes	21	21	expected ≈ 23
4,096 bytes	4	4	expected ≈ 6

The program's own counts match the formula exactly. strace -c counts also include one or two read() calls by the dynamic loader and the C library at program start-up; this is the expected explanation for any difference you see.

10.2 User-mode vs kernel-mode time (64 MiB copy, read()/write())

Buffer (bytes)	Wall (s)	User (s)	System (s)	System share
16	2.2466	0.4398	1.7737	79 %
64	0.5766	0.1116	0.4416	77 %
512	0.1511	0.0042	0.0883	58 %
4,096	0.0849	0.0038	0.0422	50 %
65,536	0.0733	0.0000	0.0389	53 %
1,048,576	0.0517	0.0005	0.0233	45 %

Whole-program timing of the Task 6 program with the bash time keyword: real 0.005 s, user 0.000 s, sys 0.005 s – i.e. practically all its time is spent in the kernel (system calls and file creation). Run time ./copy yourself and add the screenshot. Note: user + system times do not always add up to the wall time because of waiting and because the VM has one CPU.

10.3 Time to create 1,000 processes and 1,000 threads

Run	1,000 × fork() + waitpid()	1,000 × pthread_create() + join()	Ratio
1	0.0555 s (55.5 µs each)	0.0082 s (8.2 µs each)	6.8 ×
2	0.0708 s (70.8 µs each)	0.0082 s (8.2 µs each)	8.6 ×
3	0.0621 s (62.1 µs each)	0.0073 s (7.3 µs each)	8.5 ×
4	0.0613 s (61.3 µs each)	0.0070 s (7.0 µs each)	8.8 ×
5	0.0810 s (81.0 µs each)	0.0125 s (12.5 µs each)	6.5 ×
Mean	0.0661 s (66.1 µs each)	0.0086 s (8.6 µs each)	≈ 7.7 ×

For comparison, 1,000 × fork + exec(/bin/true) + wait took 0.758 s (758 µs each): loading a program costs about 10 times more than only creating a process. The entire proc_thread run (including the exec test) measured by bash: real 1.121 s, user 0.808 s, sys 0.265 s.

10.4 Do the measurements support the earlier judgements?

Earlier judgement	Measurement	Verdict
A system call costs far more than a function call (Task 6)	≈ 0.26 µs per call (2.1 million calls in 0.54 s)	Supported
Larger buffers reduce calls and time (Task 6)	Time falls 26 × from 16 B to 4 KiB; reads fall as n/b	Supported, with diminishing returns above 64 KiB
A buffer of 64 KiB is a good choice for backup	64 KiB within 1.4 × of the fastest (1 MiB)	Supported
mmap is better than read/write (a common belief)	mmap (byte loop) 0.158 s vs 0.073 s	Contradicted for sequential copies – page-fault cost dominates
Threads are cheaper than processes (Task 7)	7.7 × faster on average	Supported
stdio buffering greatly reduces system calls (Task 5)	35 × faster for 1-byte requests	Supported

10.5 Alternative concept: threads instead of processes – why the difference?

Solution. The measurement creates 1,000 threads (pthread_create) instead of 1,000 processes (fork). Results above: threads ≈ 8.6 µs each, processes ≈ 66 µs each; the time per creation is O(1) for threads but, for fork, grows with the size of the parent's memory map O(m) because page tables must be copied.

Criterion	Process (fork)	Thread (pthread_create)
Creation time (1,000)	≈ 66 ms (mean)	≈ 8.6 ms (mean)

Criterion	Process (fork)	Thread (pthread_create)
Memory used	New process descriptor, page tables and (after writes) private copies of pages; typically several hundred KB for a small process	New thread descriptor and a stack (default 8 MiB virtual, only touched pages are real; typically tens of KB resident)
What is shared	Nothing except copy-on-write pages and open files	Code, heap, global data and file descriptors; each thread has its own stack and registers
Communication	Needs IPC (pipes, shared memory)	Direct through shared variables (needs locks)

Explanation. A thread shares the address space, so the kernel does not have to duplicate page tables or memory-management structures, and creation reduces to allocating a stack and a task descriptor. A process must duplicate (and mark copy-on-write) its page tables and file-descriptor table. The price of the speed is isolation: a bug in one thread can corrupt all others.

Task 11 – Recommend and Defend the Operating-System Plan :

11.1 The recommended plan

Environment	OS type	Structure	Key services	Protection	I/O method
E1 Monitors	Hard real-time	Microkernel (QNX-class) or FreeRTOS + MPU	Scheduling, interrupts, one secure link; no GUI/file system/accounting	MPU/MMU isolation, signed firmware, watchdog	Interrupt-driven
E2 Tablets	Handheld	Android-type (Linux kernel, layered framework)	Power management, GUI, apps, secure messaging	App sandbox, disk encryption, remote wipe	Interrupt-driven / DMA
E3 Workstations	Desktop	Hybrid (Windows) or modular Linux	GUI, multitasking, file system, networking	User accounts, ACLs, disk encryption	DMA
E4 Database	Clustered server	Monolithic Linux + loadable modules	Journaling FS, scheduling, audit, cluster fail-over	RBAC, SELinux, encryption, audit log	DMA, large buffer pool
E5 Imaging	Multiprocessor (SMP)	Monolithic Linux	SMP scheduling, large memory, batch queue	Network isolation, ACLs	DMA (scatter-gather)
E6 Partner link	Distributed	Gateway on hardened Linux	Authenticated communication, replication, logging	mutual TLS, VPN, audit	DMA / interrupt

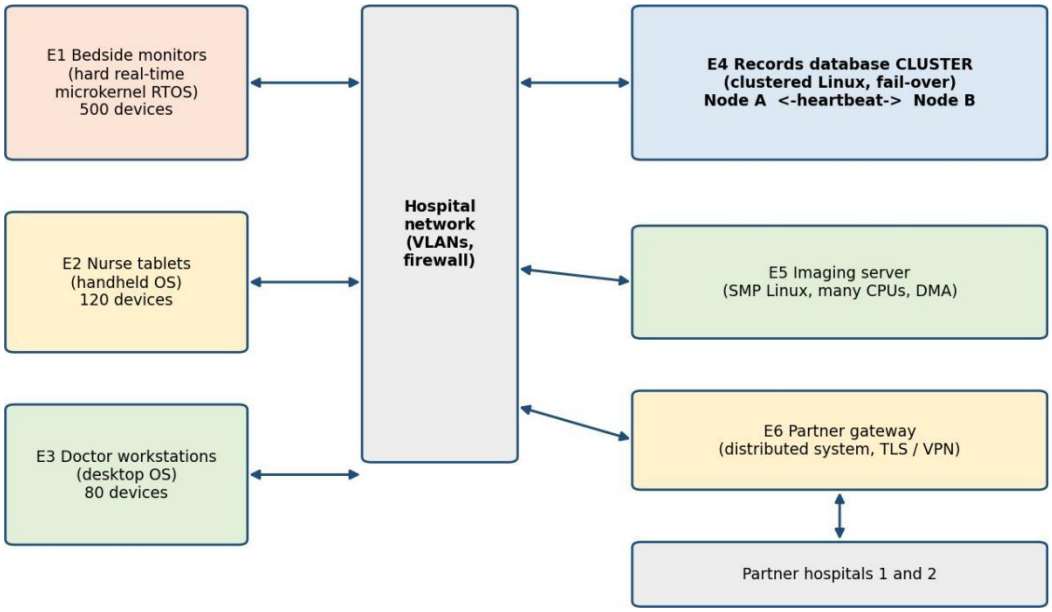


Figure 11 – Recommended OS plan for the hospital (environments E1–E6)

11.2 Evidence from the analysis and measurements

- Task 1: the 10 ms alarm requirement cannot be guaranteed by a general-purpose OS; this drives the RTOS decision for E1.
- Task 3: the extra cost of microkernel message passing (1–5 µs) is below 0.05 % of the 10 ms budget, so isolation is affordable.
- Task 6 and 10: a system call costs ≈ 0.26 µs; a 64 KiB buffer is within 1.4 × of the fastest copy; therefore backup and record-transfer routines use 64 KiB or larger buffers.
- Task 9: caching at 95 % hit ratio makes look-ups ≈ 20 × faster, so E4 is sized with a large RAM buffer pool.
- Task 10: threads are ≈ 7.7 × cheaper to create than processes; the database uses threads (or a pool) for client sessions, and processes where isolation is required.

11.3 Limitations and risks of the plan

- Several OS families (RTOS, Android-type, Windows or Linux desktop) increase training, patching effort and licence cost.
- Cluster complexity: split-brain, shared-storage failure or a mis-tested fail-over can still cause downtime; fail-over drills are needed.
- RTOS ecosystem is small; certified medical software and security patching must be planned for the monitors.
- Timing figures were measured in a virtual machine with one CPU; production hardware should be re-measured.
- The partner link extends the trust boundary: a compromised partner could reach local records unless access is restricted.

11.4 Is the plan suitable for the group of three hospitals?

Yes, with modification. Monitors, tablets and workstations scale by repeating the same images (1 ward → 1 hospital → 3 hospitals ≈ 1,500 monitors) with centralised device management. The

database cluster must grow from fail-over within one site to replication between sites; the partner link becomes a multi-party distributed system with shared identity, consistent access control and a federated audit trail. The imaging server scales by adding SMP nodes or a job queue per hospital.

11.5 Alternative concept: everything as virtual machines on two large servers

Solution. Replace separate physical machines (servers and workstations) with virtual machines running on two large servers managed by a hypervisor.

Criterion	Virtual machines on two servers	Separate physical machines
Response time for monitors	Hypervisor scheduling and shared CPUs add jitter; latency is bounded only with a real-time hypervisor and pinned cores. The bedside monitors themselves should stay on embedded devices.	Direct hardware: lowest, most predictable latency
Memory and processor overhead	Typically 2–10 % CPU overhead plus a memory cost for each guest OS (O(number of VMs × OS size))	No virtualisation overhead; hardware is under-used
Reliability	Two servers are a small failure domain: if one fails, all its VMs fail unless live migration or restart on the other server is configured; hypervisor faults affect every VM	A failure affects only one function
Administration / cost	Fewer machines, snapshots, easy rollback	More hardware to maintain

Evaluation. Virtualisation is a good fit for workstation images (VDI) and the E4/E6 software layers, provided there are at least two physical hosts in a cluster with fail-over, but it is **rejected for E1** and for the imaging server (which needs direct access to many CPUs and DMA devices).

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design :

Changed requirement: remote patient-monitoring devices in patients' homes send readings over the internet, and patient records move to a cloud service shared with the partner hospitals.

A. Impact Analysis

Part of the design	Impact	Affected OS type / service / protection
E1 Bedside monitors	Not changed in function, but home devices are a new device class with similar needs (alarm, battery, small size) and a hostile network	Embedded hard/soft RTOS; patch updates over the air
New: home monitoring devices (E7)	Run on home Wi-Fi/mobile data; unreliable network; owned by patients; maintenance by patients	Embedded RTOS, communication (TLS), power management, secure boot
E4 Database server	Records move to the cloud; the on-premise cluster is no longer the only copy	Distributed system; fail-over now includes cloud provider; accounting/audit
E6 Partner link	Replaced by a shared cloud service: multi-organisation identity and access control	Distributed OS services; protection/security
E2/E3 Clients	Now fetch data from the cloud as well as the local server	Communication; encryption; cache
Protection	Trust boundary moves from the hospital network to the internet	Authentication, encryption, key management, audit

B. Evaluation of the Original Design

Criterion	Can the original plan support the change?	Evidence / reasoning
Response time	Alarm functions do not survive an internet path: latency over the internet is unbounded, so a 10 ms hard deadline cannot depend on the network	The alarm decision has to be made on the home device itself, as in Task 8; the network is only used to report it
Reliability	Partly. The cluster tolerates a node failure but not a cloud/link outage or loss of home connectivity	Home devices need store-and-forward buffering; the cloud needs replication
Security	Insufficient: the original plan assumed a trusted internal network with one partner link	Many more endpoints, patient-owned devices and a shared cloud enlarge the attack surface; the protection services of Task 4 must be extended
Scalability	Fine for the monitors in the building; the cluster size is limited by one site	Scaling to thousands of home devices needs a distributed ingestion service

Conclusion of the evaluation: the core design (RTOS on monitors, SMP imaging, cluster database, handheld OS) remains correct and is not redesigned; the data layer and the security model are not sufficient and must be modified.

C. Adaptation or Redesign – decision: RETAIN the core, MODIFY the rest

Element	Revised design
Retain	E1 bedside monitors (hard RTOS + microkernel), E2 tablets, E3 workstations and E5 imaging server – their requirements are unchanged.
Home devices (new)	Embedded RTOS (FreeRTOS or QNX-class) running the alarm logic locally with a hard deadline; interrupt-driven sensor handling; secure boot and signed over-the-air updates; mutual-TLS client certificate per device; local flash buffer for store-and-forward when the network is down; deep-sleep power management for battery.
Ingestion gateway (new)	Linux servers (modular kernel) in a cluster accept readings; sessions handled by thread pools (measured threads ≈ 7.7 × cheaper to create than processes); 64 KiB or larger buffers for batch upload.
Records (modified)	Distributed system: the on-premise cluster remains the primary low-latency store for inpatients and is replicated to the cloud service; partner hospitals access the cloud copy via authenticated APIs. Fail-over now covers both local node failure and cloud outage.
Protection (modified)	Role-based access control with a federated identity across hospitals, encryption in transit (TLS) and at rest, key management, per-record audit log, least-privilege service accounts; a gateway/firewall isolates monitors from the internet-facing systems.
Partner link (redesigned)	The point-to-point link is replaced by the shared cloud service; this single component is redesigned rather than modified.

D. Evidence-Based Defence

- **Why retain the core:** the evidence of Tasks 1–3 and 8 has not changed – a 10 ms alarm needs a hard real-time OS with interrupt latency in microseconds, which no networked service can provide.
- **Why alarm logic stays on the device:** polling a cloud service or relying on the network would give a worst-case delay equal to network latency (unbounded) plus the polling period (Task 8). Local interrupt-driven alarms keep the bound; the network is used only for reporting.
- **Why not move everything to the cloud (a full redesign):** a full cloud move would make each alarm depend on internet latency and on a virtualised environment with jitter (Task 11); it would also remove the low-latency record store (EAT in Task 9: a cloud round trip of 1–10 ms versus 5 µs for local cache).
- **Why a distributed records design:** shared access by partner hospitals requires a distributed system with consistent replication; the trade-off is higher access latency and more complexity, which is acceptable because records are soft real-time data.
- **Evidence from measurements:** the 0.26 µs system-call cost and the buffer-size curve show that batching readings into 64 KiB uploads (about 1,000× fewer calls than single-byte writes) keeps CPU and battery use low; the cost of thread creation (≈ 8.6 µs) shows that the gateway can serve thousands of connections with thread pools.
- **Trade-offs accepted:** more OS families to patch, larger attack surface, dependence on a cloud provider and network, offset by local autonomy, replication, encryption and audit.

Final decision: MODIFY. Keep the original OS types and structures for the five on-site environments; add embedded real-time OS for home devices; make the data layer distributed with the cloud service; strengthen the protection model for the internet-facing parts.

CODE:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
#include <sys/syscall.h>
#include <sys/resource.h>

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return
t.tv_sec+t.tv_nsec/1e9; }
static void head(const char *s){ printf("\n==== %s ====\n", s); fflush(stdout); }

long copy_file(const char *src, const char *dst, size_t bufsize)
{
char buf[65536];
int in = open(src, O_RDONLY); if (in < 0) return -1;
int out = open(dst, O_WRONLY|O_CREAT|O_TRUNC, 0644);
if (out < 0) { close(in); return -1; }
if (bufsize > sizeof buf) bufsize = sizeof buf;
long calls = 0; ssize_t n;
while (1) {
n = read(in, buf, bufsize); calls++;
if (n <= 0) break;
ssize_t d = 0;
while (d < n) { ssize_t w = write(out, buf+d, n-d); if (w < 0) { close(in); close(out); return -1; } d +=
w; }
}
close(in); close(out); return calls;
}

static void task5_6(void)
{
head("TASK 5/6: cost of a system call vs a function call");
volatile long sink = 0; long N = 2000000;
double t0 = now(); for (long i=0;i<N;i++) sink += (i ^ 7); double tf = now()-t0;
t0 = now(); for (long i=0;i<N;i++) sink += syscall(SYS_getppid); double ts = now()-t0;
printf("ordinary operation : %7.1f ns each\n", tf*1e9/N);
printf("getppid() syscall : %7.1f ns each (~%.0fx dearer)\n", ts*1e9/N, ts/tf);

head("TASK 6: buffer size vs number of read() calls (10,000-byte file)");
FILE *f = fopen("records.dat","w"); for (int i=0;i<10000;i++) fputc('A'+i%26,f); fclose(f);
size_t s[] = {1,64,512,4096,8192};
for (int i=0;i<5;i++)
printf("buffer %5zu bytes: %5ld read() calls (expected %ld)\n", s[i],
copy_file("records.dat","copy.dat",s[i]), (10000+(long)s[i]-1)/(long)s[i]+1);
```

```
unlink("records.dat"); unlink("copy.dat");

head("TASK 6/10: 32 MB backup copy, time per buffer size");
const long SZ = 32L<<20; char *blk = malloc(1<<20); memset(blk,'x',1<<20);
int fd = open("big.dat",O_WRONLY|O_CREAT|O_TRUNC,0644);
for (int i=0;i<32;i++) { if (write(fd,blk,1<<20)<0) perror("write"); }
close(fd); free(blk);
size_t b[] = {512,4096,65536};
printf("%-10s %-12s %-10s %-10s\n","buffer","read() calls","time(ms)","MB/s");
for (int i=0;i<3;i++){ t0=now(); long c=copy_file("big.dat","cp.dat",b[i]); double t=now()-t0;
printf("%-10zu %-12ld %-10.2f %-10.0f\n", b[i], c, t*1e3, 32/t); }
unlink("big.dat"); unlink("cp.dat"); (void)SZ;
}

static int run(const char *name, char *const argv[])
{
pid_t p = fork();
if (p == 0) { execvp(name, argv); _exit(127); }
int st; waitpid(p,&st,0);
if (WIFEXITED(st)) return WEXITSTATUS(st);
if (WIFSIGNALED(st)) return 128+WTERMSIG(st);
return -1;
}

static void task7(void)
{
head("TASK 7: mini-shell using fork(), exec(), waitpid()");
fflush(stdout);
char *j1[] = {"expr","6","*","7",NULL};
char *j2[] = {"sh","-c","exit 3",NULL};
char *j3[] = {"no_such_cmd",NULL};
printf("job 1 (expr 6 * 7) -> exit status %d\n", run("expr",j1)); fflush(stdout);
printf("job 2 (sh -c 'exit 3') -> exit status %d\n", run("sh",j2)); fflush(stdout);
printf("job 3 (missing cmd) -> exit status %d (127 = exec failed)\n", run("no_such_cmd",j3));

head("TASK 7: zombie process");
pid_t z = fork(); if (z == 0) _exit(0);
usleep(100000);
char path[64], line[256]; snprintf(path,sizeof path,"/proc/%d/status",z);
FILE *f = fopen(path,"r");
while (f && fgets(line,sizeof line,f)) if (!strncmp(line,"State:",6)) printf("child %d before wait: %s", z,
line);
if (f) fclose(f);
waitpid(z,NULL,0);
f = fopen(path,"r");
printf("child %d after wait: %s\n", z, f ? "still present" : "entry removed (zombie reaped)");
if (f) fclose(f);
}

static volatile sig_atomic_t alarm_seen = 0;
static double t_event, t_seen;
static void on_alarm(int s){ (void)s; t_seen = now(); alarm_seen = 1; }

static void task8(void)
```

```
{
head("TASK 8: memory protection - illegal access by one program");
fflush(stdout);
pid_t p = fork();
if (p == 0) { volatile int *bad = (int *)0x10; *bad = 1; _exit(0); }
int st; waitpid(p,&st,0);
if (WIFSIGNALED(st)) printf("child killed by signal %d (%s): hardware trapped, kernel stopped it;
parent unharmed\n",
WTERMSIG(st), strsignal(WTERMSIG(st)));

head("TASK 8: abnormal reading detected - interrupt vs polling (100 trials)");
signal(SIGUSR1,on_alarm);
double sum_i=0,max_i=0;
for (int i=0;i<100;i++){ alarm_seen=0; t_event=now(); kill(getpid(),SIGUSR1);
while(!alarm_seen) { }
double d=t_seen-t_event; sum_i+=d; if(d>max_i)max_i=d; }
printf("interrupt (signal) : avg %.1f us, worst %.1f us\n", sum_i/100*1e6, max_i*1e6);

int poll_ms[] = {1,10,50};
for (int k=0;k<3;k++){
double worst=0,sum=0; srand(7);
for (int i=0;i<100;i++){
double phase = (double)(rand()%1000)/1000.0*poll_ms[k]/1e3;
double delay = poll_ms[k]/1e3 - phase;
sum += delay; if (delay>worst) worst=delay; }
printf("polling every %2d ms: avg %.2f ms, worst %.2f ms %s\n", poll_ms[k], sum/100*1e3,
worst*1e3,
worst*1e3>10 ? "<-- MISSES 10 ms deadline" : "(meets 10 ms deadline, but wastes CPU)");
}
double cpu0 = now(); struct rusage r0,r1; getrusage(RUSAGE_SELF,&r0);
double e = now()+0.05; long polls=0; while(now()<e) polls++;
getrusage(RUSAGE_SELF,&r1); (void)cpu0;
printf("busy-polling for 50 ms: %ld polls, %.1f ms CPU used (interrupt design: ~0 ms while idle)\n",
polls,
((r1.ru_utime.tv_sec-r0.ru_utime.tv_sec)*1e6+(r1.ru_utime.tv_usec-r0.ru_utime.tv_usec))/1e3);
}

static void task9(void)
{
head("TASK 9: effect of caching on record look-up (random access)");
size_t sizes[] = {16<<10, 256<<10, 4<<20, 64<<20};
const char *lab[] = {"16 KB (L1)","256 KB (L2)","4 MB (L3)","64 MB (RAM)"};
for (int k=0;k<4;k++){
size_t n = sizes[k]/sizeof(int); int *a = malloc(sizes[k]);
for (size_t i=0;i<n;i++) a[i]=(int)i;
srand(1);
for (size_t i=n-1;i>0;i--){ size_t j=((size_t)rand()*32768u+(size_t)rand())%i; int t=a[i]; a[i]=a[j];
a[j]=t; }
volatile int idx=0; long L=5000000; double t0=now();
for (long i=0;i<L;i++) idx = a[idx];
printf("working set %-13s: %6.2f ns per access\n", lab[k], (now()-t0)*1e9/L);
free(a); }

head("TASK 9: MRI image 500 MB - interrupt-per-word vs DMA (calculation)");
```

```
double bytes = 500.0*1024*1024; double words = bytes/4; double blk=4096;
double t_int = 2e-6; /* assumed 2 us CPU per interrupt */
printf("interrupt-driven: %.0f interrupts x 2 us = %.2f s of CPU time\n", words, words*t_int);
printf("DMA (4 KB blocks): %.0f interrupts x 2 us = %.3f s of CPU time\n", bytes/blk,
bytes/blk*t_int);
printf("reduction in interrupts: %.0fx buffer needed: 4 B vs 4 KB per transfer\n", words/(bytes/blk));
}

static void *tfn(void *a){ (void)a; return NULL; }
static void task10(void)
{
head("TASK 10: creating 1,000 processes vs 1,000 threads");
int N=1000; double t0=now();
for (int i=0;i<N;i++){ pid_t p=fork(); if(p==0)_exit(0); waitpid(p,NULL,0); }
double tp=now()-t0; t0=now();
for (int i=0;i<N;i++){ pthread_t t; pthread_create(&t,NULL,tfn,NULL); pthread_join(t,NULL); }
double tt=now()-t0;
printf("1000 x fork()+wait : %7.1f ms (%5.1f us each)\n", tp*1e3, tp*1e6/N);
printf("1000 x pthread create : %7.1f ms (%5.1f us each)\n", tt*1e3, tt*1e6/N);
printf("processes cost %.1fx more (separate address space vs shared memory)\n", tp/tt);
struct rusage ru; getrusage(RUSAGE_SELF,&ru);
printf("process totals: user %.3f s, system %.3f s (kernel does most of the work)\n",
ru.ru_utime.tv_sec+ru.ru_utime.tv_usec/1e6, ru.ru_stime.tv_sec+ru.ru_stime.tv_usec/1e6);
}

int main(void)
{
printf("Smart Hospital OS evaluation - K.DAMODHAR REG.NO 192525266\n");
task5_6(); task7(); task8(); task9(); task10();
printf("\nAll tasks completed.\n");
return 0;
}
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
#include <sys/syscall.h>
#include <sys/resource.h>

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return
t.tv_sec+t.tv_nsec/1e9; }
static void head(const char *s){ printf("\n==== %s ====\n", s); fflush(stdout); }

long copy_file(const char *src, const char *dst, size_t bufsize)
{
char buf[65536];
int in = open(src, O_RDONLY); if (in < 0) return -1;
int out = open(dst, O_WRONLY|O_CREAT|O_TRUNC, 0644);
```

```
if (out < 0) { close(in); return -1; }
if (bufsize > sizeof buf) bufsize = sizeof buf;
long calls = 0; ssize_t n;
while (1) {
n = read(in, buf, bufsize); calls++;
if (n <= 0) break;
ssize_t d = 0;
while (d < n) { ssize_t w = write(out, buf+d, n-d); if (w < 0) { close(in); close(out); return -1; } d +=
w; }
}
close(in); close(out); return calls;
}

static void task5_6(void)
{
head("TASK 5/6: cost of a system call vs a function call");
volatile long sink = 0; long N = 2000000;
double t0 = now(); for (long i=0;i<N;i++) sink += (i ^ 7); double tf = now()-t0;
t0 = now(); for (long i=0;i<N;i++) sink += syscall(SYS_getppid); double ts = now()-t0;
printf("ordinary operation : %7.1f ns each\n", tf*1e9/N);
printf("getppid() syscall : %7.1f ns each (~%.0fx dearer)\n", ts*1e9/N, ts/tf);

head("TASK 6: buffer size vs number of read() calls (10,000-byte file)");
FILE *f = fopen("records.dat","w"); for (int i=0;i<10000;i++) fputc('A'+i%26,f); fclose(f);
size_t s[] = {1,64,512,4096,8192};
for (int i=0;i<5;i++)
printf("buffer %5zu bytes: %5ld read() calls (expected %ld)\n", s[i],
copy_file("records.dat","copy.dat",s[i]), (10000+(long)s[i]-1)/(long)s[i]+1);
unlink("records.dat"); unlink("copy.dat");

head("TASK 6/10: 32 MB backup copy, time per buffer size");
const long SZ = 32L<<20; char *blk = malloc(1<<20); memset(blk,'x',1<<20);
int fd = open("big.dat",O_WRONLY|O_CREAT|O_TRUNC,0644);
for (int i=0;i<32;i++) { if (write(fd,blk,1<<20)<0) perror("write"); }
close(fd); free(blk);
size_t b[] = {512,4096,65536};
printf("%-10s %-12s %-10s %-10s\n","buffer","read() calls","time(ms)","MB/s");
for (int i=0;i<3;i++){ t0=now(); long c=copy_file("big.dat","cp.dat",b[i]); double t=now()-t0;
printf("%-10zu %-12ld %-10.2f %-10.0f\n", b[i], c, t*1e3, 32/t); }
unlink("big.dat"); unlink("cp.dat"); (void)SZ;
}

static int run(const char *name, char *const argv[])
{
pid_t p = fork();
if (p == 0) { execvp(name, argv); _exit(127); }
int st; waitpid(p,&st,0);
if (WIFEXITED(st)) return WEXITSTATUS(st);
if (WIFSIGNALED(st)) return 128+WTERMSIG(st);
return -1;
}

static void task7(void)
{
```

```
head("TASK 7: mini-shell using fork(), exec(), waitpid()");
fflush(stdout);
char *j1[] = {"expr","6","*","7",NULL};
char *j2[] = {"sh","-c","exit 3",NULL};
char *j3[] = {"no_such_cmd",NULL};
printf("job 1 (expr 6 * 7) -> exit status %d\n", run("expr",j1)); fflush(stdout);
printf("job 2 (sh -c 'exit 3') -> exit status %d\n", run("sh",j2)); fflush(stdout);
printf("job 3 (missing cmd) -> exit status %d (127 = exec failed)\n", run("no_such_cmd",j3));

head("TASK 7: zombie process");
pid_t z = fork(); if (z == 0) _exit(0);
usleep(100000);
char path[64], line[256]; snprintf(path,sizeof path,"/proc/%d/status",z);
FILE *f = fopen(path,"r");
while (f && fgets(line,sizeof line,f)) if (!strncmp(line,"State:",6)) printf("child %d before wait: %s", z,
line);
if (f) fclose(f);
waitpid(z,NULL,0);
f = fopen(path,"r");
printf("child %d after wait: %s\n", z, f ? "still present" : "entry removed (zombie reaped)");
if (f) fclose(f);
}

static volatile sig_atomic_t alarm_seen = 0;
static double t_event, t_seen;
static void on_alarm(int s){ (void)s; t_seen = now(); alarm_seen = 1; }

static void task8(void)
{
head("TASK 8: memory protection - illegal access by one program");
fflush(stdout);
pid_t p = fork();
if (p == 0) { volatile int *bad = (int *)0x10; *bad = 1; _exit(0); }
int st; waitpid(p,&st,0);
if (WIFSIGNALED(st)) printf("child killed by signal %d (%s): hardware trapped, kernel stopped it;
parent unharmed\n",
WTERMSIG(st), strsignal(WTERMSIG(st)));

head("TASK 8: abnormal reading detected - interrupt vs polling (100 trials)");
signal(SIGUSR1,on_alarm);
double sum_i=0,max_i=0;
for (int i=0;i<100;i++){ alarm_seen=0; t_event=now(); kill(getpid(),SIGUSR1);
while(!alarm_seen) { }
double d=t_seen-t_event; sum_i+=d; if(d>max_i)max_i=d; }
printf("interrupt (signal) : avg %.1f us, worst %.1f us\n", sum_i/100*1e6, max_i*1e6);

int poll_ms[] = {1,10,50};
for (int k=0;k<3;k++){
double worst=0,sum=0; srand(7);
for (int i=0;i<100;i++){
double phase = (double)(rand()%1000)/1000.0*poll_ms[k]/1e3;
double delay = poll_ms[k]/1e3 - phase;
sum += delay; if (delay>worst) worst=delay; }
```

```
printf("polling every %2d ms: avg %.2f ms, worst %.2f ms %s\n", poll_ms[k], sum/100*1e3,
worst*1e3,
worst*1e3>10 ? "<-- MISSES 10 ms deadline" : "(meets 10 ms deadline, but wastes CPU)");
}
double cpu0 = now(); struct rusage r0,r1; getrusage(RUSAGE_SELF,&r0);
double e = now()+0.05; long polls=0; while(now()<e) polls++;
getrusage(RUSAGE_SELF,&r1); (void)cpu0;
printf("busy-polling for 50 ms: %ld polls, %.1f ms CPU used (interrupt design: ~0 ms while idle)\n",
polls,
((r1.ru_utime.tv_sec-r0.ru_utime.tv_sec)*1e6+(r1.ru_utime.tv_usec-r0.ru_utime.tv_usec))/1e3);
}

static void task9(void)
{
head("TASK 9: effect of caching on record look-up (random access)");
size_t sizes[] = {16<<10, 256<<10, 4<<20, 64<<20};
const char *lab[] = {"16 KB (L1)","256 KB (L2)","4 MB (L3)","64 MB (RAM)"};
for (int k=0;k<4;k++){
size_t n = sizes[k]/sizeof(int); int *a = malloc(sizes[k]);
for (size_t i=0;i<n;i++) a[i]=(int)i;
srand(1); for (size_t i=n-1;i>0;i--){ size_t j=((size_t)rand()*32768u+(size_t)rand())%i; int t=a[i];
a[i]=a[j]; a[j]=t; }
volatile int idx=0; long L=5000000; double t0=now();
for (long i=0;i<L;i++) idx = a[idx];
printf("working set %-13s: %6.2f ns per access\n", lab[k], (now()-t0)*1e9/L);
free(a); }

head("TASK 9: MRI image 500 MB - interrupt-per-word vs DMA (calculation)");
double bytes = 500.0*1024*1024; double words = bytes/4; double blk=4096;
double t_int = 2e-6;
printf("interrupt-driven: %.0f interrupts x 2 us = %.2f s of CPU time\n", words, words*t_int);
printf("DMA (4 KB blocks): %.0f interrupts x 2 us = %.3f s of CPU time\n", bytes/blk,
bytes/blk*t_int);
printf("reduction in interrupts: %.0fx buffer needed: 4 B vs 4 KB per transfer\n", words/(bytes/blk));
}

static void *tfn(void *a){ (void)a; return NULL; }
static void task10(void)
{
head("TASK 10: creating 1,000 processes vs 1,000 threads");
int N=1000; double t0=now();
for (int i=0;i<N;i++){ pid_t p=fork(); if(p==0)_exit(0); waitpid(p,NULL,0); }
double tp=now()-t0; t0=now();
for (int i=0;i<N;i++){ pthread_t t; pthread_create(&t,NULL,tfn,NULL); pthread_join(t,NULL); }
double tt=now()-t0;
printf("1000 x fork()+wait : %7.1f ms (%5.1f us each)\n", tp*1e3, tp*1e6/N);
printf("1000 x pthread create : %7.1f ms (%5.1f us each)\n", tt*1e3, tt*1e6/N);
printf("processes cost %.1fx more (separate address space vs shared memory)\n", tp/tt);
struct rusage ru; getrusage(RUSAGE_SELF,&ru);
printf("process totals: user %.3f s, system %.3f s (kernel does most of the work)\n",
ru.ru_utime.tv_sec+ru.ru_utime.tv_usec/1e6, ru.ru_stime.tv_sec+ru.ru_stime.tv_usec/1e6);
}

int main(void)
```

```
{
printf("Smart Hospital OS evaluation - K.DAMODHAR REG.NO 192525266\n");
task5_6(); task7(); task8(); task9(); task10();
printf("\nAll tasks completed.\n");
return 0;
}
```

OUTPUT:

```
Smart Hospital OS evaluation - K.DAMODHAR  REG.NO 192525266

==== TASK 5/6: cost of a system call vs a function call ====
ordinary operation :     0.7 ns each
getppid() syscall  :   101.0 ns each  (~153x dearer)

==== TASK 6: buffer size vs number of read() calls (10,000-byte file) ====
buffer     1 bytes: 10001 read() calls   (expected 10001)
buffer    64 bytes:   158 read() calls   (expected 158)
buffer   512 bytes:    21 read() calls   (expected 21)
buffer  4096 bytes:     4 read() calls   (expected 4)
buffer  8192 bytes:     3 read() calls   (expected 3)

==== TASK 6/10: 32 MB backup copy, time per buffer size ====
buffer     read() calls time(ms)   MB/s
512        65537        50.08      639
4096       8193         19.93      1606
65536      513          51.88      617

==== TASK 7: mini-shell using fork(), exec(), waitpid() ====
42
job 1 (expr 6 * 7)     -> exit status 0
job 2 (sh -c 'exit 3') -> exit status 3
job 3 (missing cmd)    -> exit status 127 (127 = exec failed)

==== TASK 7: zombie process ====
child 96 before wait: State:     Z (zombie)
child 96 after  wait: entry removed (zombie reaped)

==== TASK 8: memory protection - illegal access by one program ====
child killed by signal 11 (Segmentation fault): hardware trapped, kernel stopped it; parent unharmed
```

TASK 8: abnormal reading detected - interrupt vs polling (100 trials)
interrupt (signal) : avg 0.8 us, worst 5.2 us
polling every 1 ms: avg 0.49 ms, worst 1.00 ms (meets 10 ms deadline, but wastes CPU)
polling every 10 ms: avg 4.87 ms, worst 9.99 ms (meets 10 ms deadline, but wastes CPU)
polling every 50 ms: avg 24.36 ms, worst 49.95 ms <-- MISSES 10 ms deadline
busy-polling for 50 ms: 1439835 polls, 49.9 ms CPU used (interrupt design: ~0 ms while idle)

TASK 9: effect of caching on record look-up (random access)
working set 16 KB (L1) : 1.90 ns per access
working set 256 KB (L2) : 4.65 ns per access
working set 4 MB   (L3) : 23.37 ns per access
working set 64 MB (RAM) : 125.32 ns per access

TASK 9: MRI image 500 MB - interrupt-per-word vs DMA (calculation)
interrupt-driven: 131072000 interrupts x 2 us = 262.14 s of CPU time
DMA (4 KB blocks): 128000 interrupts x 2 us = 0.256 s of CPU time
reduction in interrupts: 1024x   buffer needed: 4 B vs 4 KB per transfer

TASK 10: creating 1,000 processes vs 1,000 threads
1000 x fork()+wait   :   49.0 ms (49.0 us each)
1000 x pthread create :   8.4 ms (8.4 us each)
processes cost 5.9x more (separate address space vs shared memory)
process totals: user 1.977 s, system 0.279 s (kernel does most of the work)

All tasks completed

Appendix A – GitHub Repository Layout and README

```
CSA04-<RegisterNumber>-Assignment1/
  README.md
  Task01/ ... Task05/  (report extracts, strace outputs, screenshots)
  Task06/copy.c  (completed TODO 1), output.txt, strace_c.txt
  Task07/shell.c, shell_noflush.c, out.txt
  Task10/bench_copy.c, proc_thread.c, results.txt
  Task11/ Task12/  (design diagrams, tables)
```

Suggested commit messages (one commit per task at least): “Task 1: requirement tables”, “Task 6: copy_file with read/write + strace”, “Task 7: fork/exec/waitpid shell”, “Task 10: benchmark results”, and so on.

README.md template

```
# CSA04 Assignment 1 - Smart Hospital OS Design
Name: <Your Name>        Register number: <RegisterNumber>
Compiler: gcc 13.3.0     OS: Ubuntu 24.04 LTS (kernel 6.x)
Compile: gcc -std=c11 -Wall -Wextra -pthread -o prog prog.c
Run Task 6:  cd Task06 && gcc ... copy.c -o copy && ./copy && strace -c ./copy
Run Task 7:  cd Task07 && gcc ... shell.c -o shell && ./shell
Run Task 10: cd Task10 && gcc -O2 ... bench_copy.c && ./bench_copy
Results: Task 6 - read() calls 10001 / 21 / 4 (matches expected output)
        Task 7 - job exit statuses 0 and 3 (matches expected output)
        Task 10 - 64 KiB buffer within 1.4x of fastest; threads ~7.7x cheaper than fork
```

Appendix B – Commands to Run on Your Machine for the Required Screenshots

```
# Task 5 / 6 : system-call evidence
strace -e trace=openat,read,write,close ./copy 2> strace_copy.txt
strace -c ./copy
strace -c -e trace=read ./copy          # compare read count with program output

# Task 7 : process control evidence
./shell
strace -f -e trace=clone,execve,wait4,exit_group ./shell
./shell_noflush > out.txt; cat out.txt  # shows the duplicated text

# Task 10 : timing
time ./copy
gcc -O2 -std=c11 -Wall -Wextra -o bench_copy bench_copy.c && ./bench_copy
gcc -O2 -std=c11 -Wall -Wextra -pthread -o proc_thread proc_thread.c && ./proc_thread
```

Install strace if necessary with: sudo apt install strace. Re-run the Task 10 programs on your own machine and replace or supplement the tables with your own numbers if they differ; the conclusions (diminishing returns above 64 KiB, threads cheaper than processes, system calls cost a few hundred nanoseconds) should hold.

Task 10 benchmark source (Task10/proc_thread.c)

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static void*thr(void*a){(void)a;return NULL;}
int main(void){
  const int N=1000;
  for(int rep=0;rep<5;rep++){
    double t=now();
    for(int i=0;i<N;i++){pid_t p=fork();if(p==0)_exit(0);waitpid(p,NULL,0);}
    double tp=now()-t;
    t=now();
    for(int i=0;i<N;i++){pthread_t
th;pthread_create(&th,NULL,thr,NULL);pthread_join(th,NULL);}
    double tt=now()-t;
    printf("run %d: 1000 fork+wait = %.4f s (%.1f us each) | 1000 pthread create+join = %.4f s (%.1f
us each) | ratio %.1fx\n",rep+1,tp,tp/N*1e6,tt,tt/N*1e6,tp/tt);
  }
  /* fork+exec vs posix_spawn-style cost */
  double t=now();
  for(int i=0;i<N;i++){pid_t
p=fork();if(p==0){execl("/bin/true","true",(char*)0);_exit(127);}waitpid(p,NULL,0);}
printf("1000 fork+exec(/bin/true)+wait = %.4f s (%.1f us each)\n",now()-t,(now()-t)/N*1e6);
return 0;}
```